Instructivo de Práctica — Sesión 4: PlacesController compartido con GetX

> 📌 **Qué vas a entregar:** al terminar esta práctica, el estado de la lista de lugares va a vivir en un único `PlacesController` de GetX en vez de quedar atrapado dentro de `HomeScreen` — agregar un lugar desde el formulario se refleja en Inicio sin recargar nada a mano, y la pantalla muestra un contador y un aviso construidos con lo visto en la teoría. Es un refactor: no se agrega ninguna pantalla nueva, se reorganiza la lógica que ya existe.

Objetivo

Incorporar un `PlacesController` de GetX que centraliza la lista de lugares y su estado de carga, y refactorizar `HomeScreen` para que lo lea en vez de mantener su propio estado local.

Requisitos previos

- Entorno funcionando desde la Sesión 1, con `exploraec` corriendo.
- Haber completado la Sesión 3 (tema de marca, estados de carga/vacío/error con `FutureBuilder`, layout responsivo).

Tiempo estimado

Total: 55 minutos

Paso	Descripción	Tiempo
0	Recap de la Sesión 3: el problema que vamos a resolver	6 min
1	Traer el código de la Sesión 4 y explorar lo que ya viene resuelto	8 min
2	Conectar el controller a la carga de lugares	10 min
3	Refactorizar `HomeScreen` a `Obx`	13 min
4	Verificar la navegación con GetX de punta a punta	8 min

Paso	Descripción	Tiempo
5	Aplicar la teoría: estado derivado, aviso sin `context` y un worker	10 min
6 *(opcional)*	Favoritos e idioma con GetX — no cuenta dentro de los 55 min	15 min

---

Paso 0 — Recap de la Sesión 3: el problema que vamos a resolver (6 min)

Por qué se hace

Antes de traer código nuevo conviene ver, en tu propia app, el problema que resuelve GetX: hoy todo el estado de la lista vive dentro de `HomeScreen`. Este paso se hace **con tu proyecto tal como quedó en la Sesión 3**, antes de traer la rama de hoy.

Instrucciones

1. En tu proyecto `exploraec`, confirmar que el trabajo de la Sesión 3 está guardado:

```
git status
```

Si aparecen cambios sin guardar, hacer `git add -A && git commit -m "Sesión 3"`. 2. Ejecutar la app (`flutter run`) y repasar lo construido: el tema de marca, el menú " ⋮ " de la barra superior (normal / vacío / error) y las pantallas de carga, vacío y error. 3. Tocar el botón "+", agregar un lugar y guardar. Observar dos cosas: al volver, Inicio muestra de nuevo "Buscando lugares cercanos..." (recarga **toda** la lista) y el lugar nuevo queda al final de la lista (hay que desplazarse). 4. Abrir `lib/screens/home_screen.dart` y ubicar las cuatro piezas de estado que viven dentro de `HomeScreen`: `_futuroLugares`, las banderas `_modoDebugError` / `_modoDebugVacio`, el método `_cargar()` con su `setState`, y el `await Navigator.push(...)` seguido de `_cargar()`. 5. Responder en una frase: ¿podrían la pestaña Mapa o Favoritos leer `_futuroLugares`? ¿Por qué no?

Resultado esperado

Se pueden nombrar las cuatro piezas de estado atrapadas en `HomeScreen` y explicar por qué otra pantalla no puede verlas. Ese es el problema que resuelve el `PlacesController` de hoy.

Errores comunes

- **`git status` muestra cambios de la Sesión 3 sin guardar**. *Solución:* guardarlos con un commit antes de seguir: el Punto de partida de abajo sobrescribe `lib/`.
- **La app no compila antes de empezar**. *Solución:* confirmar que la práctica de la Sesión 3 quedó completa (`flutter analyze` sin errores) antes de continuar.

✅ Antes de continuar, confirma que...

- ☐ El trabajo de la Sesión 3 está guardado en un commit.
- ☐ Viste que agregar un lugar recarga toda la lista al volver.
- ☐ Ubicaste `_futuroLugares`, `_cargar()` y el `Navigator.push` en `home_screen.dart`.

Punto de partida

```
git fetch starter
git checkout starter/sesion-04 -- lib pubspec.yaml PLACEHOLDERS.md
flutter pub get
```

Esto trae, ya escritos, `lib/controllers/places_controller.dart`, `lib/bindings/places_binding.dart`, `main.dart` con `GetMaterialApp`, y la navegación de `place_card.dart` / `add_place_screen.dart` ya migrada a `Get.to` / `Get.back` — además de `get: ^4.6.6` en `pubspec.yaml`. Ver `PLACEHOLDERS.md` para el detalle de qué llega completo y qué queda por descomentar.

Paso 1 — Explorar lo que ya viene resuelto (8 min)

Por qué se hace

GetX introduce piezas nuevas (`GetxController`, `Bindings`, `GetView`) que conviene leer antes de tocar código — la rama de hoy ya las trae escritas para que la práctica se enfoque en el refactor central, no en escribir boilerplate de instalación.

Instrucciones

1. Con los archivos ya traídos a tu proyecto `exploraec` (Punto de partida), abrir `lib/`

`controllers/places_controller.dart` y leerlo completo: identificar `RxList<Place> lugares`, `Rx<EstadoCarga> estado` y `RxString mensajeError`, y el bloque `TODO(sesion-04)` dentro de `cargarLugares()` (se resuelve en el Paso 2).

2. Abrir `lib/bindings/places_binding.dart`: confirmar que `Get.put(PlacesController())` registra el controller una sola vez.
3. Abrir `lib/main.dart`: confirmar que `ExploraEcApp` ya usa `GetMaterialApp` con `initialBinding: PlacesBinding()`.
4. Abrir `lib/widgets/place_card.dart` y `lib/screens/add_place_screen.dart`: confirmar que la navegación ya usa `Get.to` / `Get.back` en vez de `Navigator.push` / `Navigator.pop`.
5. Ejecutar la app y confirmar que compila (todavía sin ningún cambio visual — `HomeScreen` muestra "Pendiente de conectar con Obx"):

```
flutter run
```

Resultado esperado

Se puede explicar, en una frase cada una: qué hace `PlacesController`, qué hace `PlacesBinding`, y por qué `main.dart` cambió de `MaterialApp` a `GetMaterialApp`. La app corre, mostrando el texto de aviso en Inicio; las pestañas Mapa y Favoritos siguen siendo los placeholders de la Sesión 2.

Errores comunes

- **`Target of URI doesn't exist: 'package:get/get.dart'`**. *Solución:* confirmar que el Punto de partida trajo el `pubspec.yaml` de la rama `sesion-04` (incluye `get: ^4.6.6`) y correr `flutter pub get` de nuevo.

✅ Antes de continuar, confirma que...

- ☐ Puedes explicar qué hace `PlacesController`, `PlacesBinding`, y por qué `main.dart` usa `GetMaterialApp`.
- ☐ La app compila y corre (con el texto "Pendiente de conectar con Obx" en Inicio).

Paso 2 — Conectar el controller a la carga de lugares (10 min)

Por qué se hace

`PlacesController` ya existe, pero `cargarLugares()` todavía no llama a nada — solo devuelve una lista vacía. Este paso lo conecta con `fetchLugaresSimulado`, la misma función simulada de la Sesión 3 que hoy llama `HomeScreen` desde su propio `State`.

Instrucciones

1. En `lib/controllers/places_controller.dart`, ubicar el comentario `// TODO(sesion-04): borra la línea de abajo y descomenta el bloque completo. (Paso 2 — conectar el controller a la carga de lugares)` y leer el comentario `// Por qué:` que lo acompaña.
2. Borrar las dos líneas activas debajo (`lugares.value = []; estado.value = EstadoCarga.exito;`).
3. Descomentar el bloque `try { ... } catch (e) { ... }` completo que sigue.

Resultado esperado

El código de `cargarLugares()` ahora llama a `fetchLugaresSimulado(...)` y actualiza los tres estados (`cargando`, `exito`, `error`), igual que hacía `HomeScreen._cargar()` en la Sesión 3 — pero la app **todavía no muestra nada nuevo en pantalla**, porque `HomeScreen` no lee el controller hasta el Paso 3.

Errores comunes

- **Dejar las dos versiones activas a la vez** (la línea vacía y el bloque `try` sin comentar). *Solución:* revisar que la línea `lugares.value = [];` quedó borrada, no solo comentada — dos asignaciones a `estado.value` en la misma función es una señal de que algo quedó a medias.

✅ Antes de continuar, confirma que...

- ☐ `cargarLugares()` llama a `fetchLugaresSimulado(...)` dentro de un `try` / `catch`.
- ☐ Ya no están las dos líneas viejas (`lugares.value = [];` y `estado.value = EstadoCarga.exito;`) y el bloque `try` / `catch` quedó descomentado (el comentario `TODO` puede quedarse).

Paso 3 — Refactorizar `HomeScreen` a `Obx` (13 min)

Por qué se hace

Con el controller ya conectado, falta que la pantalla lo lea. Este paso reemplaza el texto de aviso por `Obx`, que reconstruye la pantalla sola cuando el controller cambia, y elimina el `State`, el `Future` local y las banderas de simulación que hoy viven dentro de `HomeScreen`.

Instrucciones

1. En `lib/screens/home_screen.dart`, ubicar `// TODO(sesion-04): ... (Paso 3 — reactividad con Obx)` y su comentario `// Por qué:`, borrar la línea `body: const Center(child: Text('Pendiente de conectar con Obx'))`, y descomentar el `body: Obx(() { ... })` completo.
2. Confirmar que `HomeScreen` extiende `GetView<PlacesController>` y no `StatefulWidget`.
3. Ejecutar `flutter run` (o hot restart si ya estaba corriendo, ya que `main.dart` cambió a `GetMaterialApp`).

Resultado esperado

Inicio vuelve a mostrar lugares (loading → lista, igual que en la Sesión 3), y el menú de simulación (normal/vacío/error) sigue funcionando — ahora a través del controller. Cada tarjeta ya muestra un corazón, pero todavía no responde al toque: se conecta en el Paso 6 (opcional).

Errores comunes

- **`"PlacesController" not found. You need to call "Get.put(PlacesController())" or "Get.lazyPut(...)"`**. *Solución:* revisar que `main.dart` use `GetMaterialApp` con `initialBinding: PlacesBinding()` (ya debería venir así desde el Punto de partida) — es el error más común de esta práctica, y casi siempre indica que se sobrescribió `main.dart` con una versión de otra sesión.
- **La pantalla no se actualiza aunque el controller sí cambió**. *Solución:* revisar que el `body` quedó envuelto en `Obx(() => ...)` y no en un `Builder` normal — un `Obx` mal cerrado (paréntesis de más o de menos) puede compilar pero no reaccionar.
- **Hot reload no refleja el cambio**. *Solución:* `GetMaterialApp` es un cambio en la raíz del árbol de widgets — usar hot restart (`R`), no hot reload (`r`).

✅ Antes de continuar, confirma que...

- ☐ `HomeScreen` extiende `GetView<PlacesController>`.
- ☐ Inicio muestra los tres estados (cargando, lista y, con el menú de simulación, vacío y

error).

- ☐ Ya no está el texto "Pendiente de conectar con Obx" y el `body: Obx(...)` quedó descomentado (el comentario `TODO` puede quedarse).

Paso 4 — Verificar la navegación con GetX de punta a punta (8 min)

Por qué se hace

`place_card.dart` y `add_place_screen.dart` ya llegaron con la navegación migrada a `Get.to` / `Get.back` y con `agregarLugar` (Punto de partida) — este paso confirma que, junto con el controller y el `Obx` de hoy, el flujo completo funciona de punta a punta.

Instrucciones

1. Abrir Inicio, tocar el botón "+", agregar un lugar nuevo y guardar.
2. Confirmar que vuelve a Inicio **sin** volver a mostrar "Buscando lugares cercanos..." (ya no hay recarga) y que el lugar nuevo aparece en la lista: se agrega al final, así que hay que desplazarse hasta él.
3. Tocar una tarjeta y confirmar que el Detalle se abre con `Get.to`; volver con el botón de retroceso.

Resultado esperado

Agregar un lugar desde el formulario lo muestra de inmediato en Inicio, sin ninguna acción adicional del usuario ni código de recarga en `HomeScreen`.

Errores comunes

- `"You are trying to use contextless navigation without a GetMaterialApp or Get.key"`. *Solución:* confirmar que `main.dart` usa `GetMaterialApp`, no `MaterialApp`.
- **El lugar nuevo no aparece en Inicio**. *Solución:* confirmar que `AddPlaceScreen` llama a `Get.find<PlacesController>().agregarLugar(...)` y que `HomeScreen` lee `controller.lugares` dentro de su `Obx`.

✅ Antes de continuar, confirma que...

- ☐ Un lugar agregado desde el formulario aparece de inmediato en Inicio.
- ☐ Nadie tiene código con `Navigator.push` / `MaterialPageRoute` restante (`grep -rn`

`"Navigator\." lib/`: solo deben aparecer líneas de comentarios).

Paso 5 — Aplicar la teoría: estado derivado, aviso sin `context` y un worker (10 min)

Por qué se hace

Con el controller funcionando, este paso aplica tres ideas de la teoría de hoy sobre el mismo código: un getter derivado (que el controller calcula sin guardar un dato aparte), un aviso sin `context` con `Get.snackbar`, y un worker (`ever`) que reacciona a un cambio de estado.

Instrucciones

1. **Estado derivado.** En `lib/controllers/places_controller.dart`, antes de la llave que cierra la clase, agregar:

```
  /// Estado derivado: se calcula a partir de `lugares`, no se guarda aparte.
  int get total => lugares.length;
```

Luego, en `lib/screens/home_screen.dart`, reemplazar `title: const Text('ExploraEC'),` de la `AppBar` por:

```
        title: Obx(() => Text('ExploraEC (${controller.total})')),
```

2. **Aviso sin `context`.** En `lib/screens/add_place_screen.dart`, justo debajo de `Get.back();`, agregar:

```
                    Get.snackbar('Lugar agregado', 'Ya aparece en Inicio');
```

3. **Worker.** En `places_controller.dart`, dentro de `onInit()`, entre `super.onInit();` y `cargarLugares();`, agregar:

```
    ever(estado, (EstadoCarga e) {
      if (e == EstadoCarga.error) {
        Get.snackbar('Error', mensajeError.value);
      }
    });
```

4. Guardar y hacer hot restart. Probar: (a) el título muestra `ExploraEC (6)`; (b) agregar un lugar sube el contador a 7 y muestra el aviso "Lugar agregado"; (c) en el menú " ⋮ " elegir "Simular: error" y comprobar que además de `ErrorView` aparece el aviso "Error".

Resultado esperado

El título de Inicio cuenta los lugares y se actualiza solo; guardar un lugar muestra un aviso sin haber pasado `context`; el estado de error dispara además un aviso desde el propio controller.

Errores comunes

- **El contador no cambia al agregar un lugar**. *Solución:* confirmar que el título quedó dentro de `Obx(() => ...)` y que `total` lee `lugares` (un getter que no lee ningún observable no notifica).
- **`Get.snackbar` no muestra nada o lanza un error de overlay**. *Solución:* confirmar que `main.dart` usa `GetMaterialApp`, no `MaterialApp`.
- **El aviso de error aparece varias veces**. *Solución:* revisar que `ever(...)` esté en `onInit()` y no dentro de un método que se llame en cada recarga.

✅ Antes de continuar, confirma que...

- ☐ El título de Inicio muestra `ExploraEC (N)` y se actualiza al agregar un lugar.
- ☐ Guardar un lugar muestra el aviso "Lugar agregado".
- ☐ "Simular: error" muestra además el aviso "Error".

Paso 6 (opcional) — Favoritos e idioma con GetX (15 min)

> Este paso **no cuenta dentro de los 55 minutos** ni bloquea la Sesión 5. Hazlo si terminaste antes que el resto o como repaso. Sigue la misma mecánica de los pasos anteriores: leer el `TODO`, borrar lo que indica y descomentar el bloque.

Por qué se hace

Hasta ahora el `PlacesController` guarda una sola lista. Este paso le agrega dos usos nuevos del mismo patrón: **(A)** una segunda lista reactiva (`favoritos`) que dos pantallas distintas leen a la vez, y **(B)** el idioma de la app, que también es estado global: `Get.updateLocale` cambia el idioma de toda la interfaz sin `setState` ni `context` . Los dos ya vienen armados en la rama de hoy (`favoritos` , `esFavorito` , `totalFavoritos` , el corazón de `PlaceCard` , la pestaña Favoritos y `lib/i18n/app_translations.dart`); solo falta conectarlos.

Paso 6A — Favoritos en memoria (7 min)

Instrucciones

1. En `lib/controllers/places_controller.dart` , ubicar `// TODO(sesion-04): OPCIONAL — ... (Paso 6A — favoritos en memoria)` y leer su comentario `// Por qué:` .
2. Borrar la línea `void alternarFavorito(Place lugar) {}` y descomentar el bloque completo que sigue.
3. Guardar, hacer hot reload (`r`) y tocar el corazón de dos o tres tarjetas en Inicio.
4. Abrir la pestaña **Favoritos** y confirmar que dice "Favoritos marcados: N". Volver a Inicio, quitar uno y revisar de nuevo.

Resultado esperado

El corazón se pone rojo al tocarlo y vuelve a gris al tocarlo otra vez. La pestaña Favoritos muestra el número correcto sin que Inicio le avise nada: las dos pantallas leen el mismo `PlacesController` . Si cierras la app —o haces hot restart (`R`)—, los favoritos se pierden (viven solo en memoria): la Sesión 7 los guarda con Hive sin cambiar los nombres de estos métodos.

Errores comunes

- **El corazón no cambia al tocarlo**. *Solución:* confirmar que se borró `void alternarFavorito(Place lugar) {}` y se descomentó el bloque completo: con el método vacío, tocar el corazón no hace nada.
- `'alternarFavorito' is already declared in this scope` (en la terminal de `flutter run` , al hacer hot reload). *Solución:* quedaron las dos versiones; borrar la línea vacía `void alternarFavorito(Place lugar) {}` , no solo comentarla.

✅ Antes de continuar, confirma que...

- ☐ El corazón se marca y desmarca en Inicio.
- ☐ La pestaña Favoritos muestra cuántos lugares marcaste.

Paso 6B — Idioma español/inglés con GetX (8 min)

Instrucciones

1. En `lib/main.dart`, descomentar el `import 'i18n/app_translations.dart';` de la parte superior (`TODO ... OPCIONAL`, no hay nada que borrar).
2. En el mismo archivo, dentro de `GetMaterialApp`, descomentar las tres líneas `translations`, `locale` y `fallbackLocale` (no hay nada que borrar).
3. En `main.dart`, en la barra inferior, borrar el bloque `items: const [...]` y descomentar el bloque `items: [...]` que usa `'inicio'.tr`, `'mapa'.tr` y `'favoritos'.tr`.
4. En `lib/screens/home_screen.dart`, descomentar el `IconButton` de idioma dentro de `actions` (no hay nada que borrar).
5. En el mismo archivo, en el menú " ⋮ ", borrar el bloque `itemBuilder: ... const [...]` y descomentar el que usa `'sim_normal'.tr`, `'sim_vacio'.tr` y `'sim_error'.tr`.
6. Hacer hot restart (`R`; `GetMaterialApp` cambió en la raíz). Tocar el ícono de traducir (Aa) de la barra superior y observar la barra inferior y el menú " ⋮ ". Tocarlo otra vez para volver a español.
7. Con la app en inglés, marcar un favorito y abrir la pestaña Favoritos: el estado de los favoritos no cambia con el idioma.

Resultado esperado

Al tocar el botón de idioma, "Inicio / Mapa / Favoritos" pasan a "Home / Map / Favorites" y el menú " ⋮ " a "Simulate: normal / empty / error", todo al instante y sin reiniciar la app. El título `ExploraEC (N)`, las tarjetas y los favoritos no cambian.

Errores comunes

- **Aparecen textos como `inicio` o `sim_vacio` en vez de "Inicio"**. *Solución:* `.tr` no encontró la clave: revisar que las tres líneas de `GetMaterialApp` (`translations`, `locale`, `fallbackLocale`) estén descomentadas y hacer hot restart.
- **`Duplicated named argument 'items'`** (o `itemBuilder`). *Solución:* quedaron las dos

versiones a la vez; borrar el bloque `const [...]` viejo completo, no solo descomentar el nuevo.

- `Extension methods can't be used in constant expressions`. *Solución:* se escribió `.tr` dentro de una lista que sigue siendo `const`. `.tr` depende del idioma activo, así que no puede ser constante: usar el bloque sin `const` que ya viene comentado.
- `The method 'AppTranslations' isn't defined for the type 'ExploraEcApp'`. *Solución:* falta descomentar el `import` del Paso 1.
- **Siguen apareciendo claves como `inicio` aunque ya descomentaste todo**. *Solución:* hot reload (`r`) no basta; hacer hot restart (`R`), porque `translations` y `locale` se leen al construir la raíz.
- **El idioma vuelve a español al cerrar la app, y las pantallas Agregar lugar, Detalle y Favoritos siguen en español**. *Solución:* es lo esperado: el idioma no se guarda (la Sesión 7 enseña a guardar datos) y solo se tradujeron la barra inferior y el menú. Ver el mini-reto.

✅ Antes de continuar, confirma que...

- ☐ El botón de idioma alterna entre español e inglés.
- ☐ La barra inferior y el menú " ⋮ " cambian de idioma.
- ☐ Los favoritos marcados se conservan al cambiar de idioma.

Mini-retos *(sin solución escrita)*

1. **Traducir Agregar lugar.** El diccionario ya trae las claves `agregar_lugar`, `nombre_lugar`, `categoria`, `descripcion`, `guardar`, `lugar_agregado` y `lugar_agregado_msg`. Úsalas con `.tr` en `add_place_screen.dart`, incluido el `Get.snackbar` del Paso 5. Pista: quita los `const` donde haga falta.
2. **Contador en la barra.** Muestra `totalFavoritos` como un `Badge` sobre el ícono de Favoritos de la barra inferior. Pista: necesita un `Obx`, porque el número cambia.

> **Nota sobre el enfoque:** `.tr` de GetX es la forma rápida de traducir una app. Para una app de producción, Flutter recomienda `flutter_localizations` con `intl` y archivos ARB (soporta plurales, fechas y textos propios de Material, como el tooltip del botón de retroceso, que aquí sigue en inglés).

Entregable de la práctica

- `PlacesController` conectado a la carga de lugares, con los tres estados.
- `HomeScreen` leyendo el controller con `Obx`, sin `StatefulWidget` ni recarga manual.
- Navegación completa verificada con `Get.to` / `Get.back`.
- Contador derivado (`total`), aviso con `Get.snackbar` y un worker `ever` aplicados sobre el controller.
- *(Opcional)* Favoritos en memoria compartidos entre Inicio y la pestaña Favoritos, y cambio de idioma español/inglés con `Get.updateLocale`.

Conexión con la siguiente sesión

En la Sesión 5 se reemplaza el placeholder de la pestaña Mapa por un mapa real (`flutter_map`) con la posición del usuario. Esa pantalla va a nacer leyendo `controller.lugares` del mismo `PlacesController` de hoy — es indispensable llegar con el controller funcionando correctamente.